

Algoritmi e Strutture Dati

Analisi di algoritmi Ricorrenze e analisi per livelli

Alberto Montresor

Università di Trento

2026/09/10

This work is licensed under a Creative Commons
Attribution-ShareAlike 4.0 International License.



Dall'algoritmo alla ricorrenza

Relazione di ricorrenza

Il costo di un algoritmo ricorsivo dipende dal costo di una o più chiamate su input più piccoli. Questa dipendenza è descritta da una **relazione di ricorrenza**.

Merge Sort

$$T(n) = \begin{cases} T(\lfloor n/2 \rfloor) + T(\lceil n/2 \rceil) + \Theta(n) & n > 1, \\ \Theta(1) & n \leq 1. \end{cases}$$

Dall'algoritmo alla ricorrenza

Forma chiusa

L'obiettivo non è necessariamente trovare una formula esatta, ma una **forma chiusa** che esprima l'ordine di crescita.

Merge Sort

$$T(n) = \Theta(n \log n).$$

Oltre l'analisi di algoritmi

Le ricorrenze descrivono anche quantità definite tramite istanze più piccole dello stesso problema.

Un bambino scende una scala; ad ogni passo può scendere 1, 2, 3 oppure 4 scalini. Sia $M(n)$ è il numero di modi per scendere n scalini. $M(n)$ può essere calcolato nel modo seguente:

$$M(n) = \begin{cases} 0 & n < 0, \\ 1 & n = 0, \\ \sum_{j=1}^4 M(n-j) & n > 0. \end{cases}$$

Numeri di tetranacci: 1, 1, 2, 4, 8, 15, 29, 56, 108, 208, 401, 773, ...

Tecniche di analisi

Tre metodi complementari

- **Analisi per livelli**: rende visibili numero e costo delle chiamate
- **Sostituzione**: formula un'ipotesi e la dimostra per induzione
- **Teoremi degli esperti**: risolvono direttamente famiglie di ricorrenze

Analisi per livelli

Le quantità da calcolare a ogni livello

- numero di chiamate e dimensione dei sottoproblemi
- costo non ricorsivo di una singola chiamata
- costo complessivo del livello

Semplificazione

- Nel seguito, non utilizzeremo valori costanti come c e d per descrivere le costanti moltiplicative del lavoro di ricombinazione e del caso base
- Utilizzeremo il valore 1 per semplificare i calcoli
- Le costanti moltiplicative vengono assorbite dalle notazioni asintotiche

Una sola chiamata: espansione lineare

$$T(n) = \begin{cases} T(n/2) + n & n > 1, \\ 1 & n \leq 1. \end{cases}$$

$$\begin{aligned} T(n) &= T(n/2) + n \\ &= T(n/4) + \frac{n}{2} + n \\ &= T(n/8) + \frac{n}{4} + \frac{n}{2} + n \end{aligned}$$

$$\vdots$$

$$= T(1) + \sum_{i=0}^{\log n - 1} \frac{n}{2^i}.$$

Assunzioni

$n = 2^k$ con k intero

$k = \log n.$

Più chiamate: organizziamo il calcolo

Stessa struttura ricorsiva

Confrontiamo tre ricorrenze con quattro sottoproblemi di dimensione $n/2$:

$$4T(n/2) + n, \quad 4T(n/2) + n^2, \quad 4T(n/2) + n^3.$$

La domanda decisiva

Come cambia il **costo complessivo di un livello** quando scendiamo nell'albero delle chiamate?

- il numero di chiamate cresce di un fattore 4;
- la dimensione di ciascun sottoproblema si dimezza;
- il lavoro non ricorsivo determina quale effetto prevale.

Primo caso: $T(n) = 4T(n/2) + n$

Livello	N. chiamate	Dimensione	Costo per chiamata	Costo del livello
0	1	n	n	n
1	4	$n/2$	$n/2$	$n \cdot 2$
2	4^2	$n/2^2$	$n/2^2$	$n \cdot 4$
i	4^i	$n/2^i$	$n/2^i$	$n \cdot 2^i$
k	4^k	1	1	n^2

Primo caso: $T(n) = 4T(n/2) + n$

Livello	N. chiamate	Dimensione	Costo per chiamata	Costo del livello
0	1	n	n	n
1	4	$n/2$	$n/2$	$n \cdot 2$
2	4^2	$n/2^2$	$n/2^2$	$n \cdot 4$
i	4^i	$n/2^i$	$n/2^i$	$n \cdot 2^i$
k	4^k	1	1	n^2

$$T(n) = n \sum_{i=0}^{k-1} 2^i + n^2 = n(n-1) + n^2.$$

$$k = \log n, 4^{\log n} = n^2$$

$$T(n) = \Theta(n^2)$$

Il costo cresce: **dominano gli ultimi livelli** e le sole foglie costano già n^2 .

Secondo caso: $T(n) = 4T(n/2) + n^2$

Livello	N. chiamate	Dimensione	Costo per chiamata	Costo del livello
0	1	n	n^2	n^2
1	4	$n/2$	$(n/2)^2$	n^2
2	4^2	$n/2^2$	$(n/2^2)^2$	n^2
i	4^i	$n/2^i$	$(n/2^i)^2$	n^2
k	4^k	1	1	n^2

Secondo caso: $T(n) = 4T(n/2) + n^2$

Livello	N. chiamate	Dimensione	Costo per chiamata	Costo del livello
0	1	n	n^2	n^2
1	4	$n/2$	$(n/2)^2$	n^2
2	4^2	$n/2^2$	$(n/2^2)^2$	n^2
i	4^i	$n/2^i$	$(n/2^i)^2$	n^2
k	4^k	1	1	n^2

$$T(n) = \sum_{i=0}^{k-1} n^2 + n^2 = n^2 \log n + n^2 = \Theta(n^2 \log n).$$

L'aumento del numero di chiamate compensa esattamente la diminuzione del costo per chiamata: **tutti i livelli hanno lo stesso costo.**

Terzo caso: $T(n) = 4T(n/2) + n^3$

Livello	N. chiamate	Dimensione	Costo per chiamata	Costo del livello
0	1	n	n^3	n^3
1	4	$n/2$	$(n/2)^3$	$n^3/2$
2	4^2	$n/2^2$	$(n/2^2)^3$	$n^3/2^2$
i	4^i	$n/2^i$	$(n/2^i)^3$	$n^3/2^i$
k	4^k	1	1	n^2

Terzo caso: $T(n) = 4T(n/2) + n^3$

Livello	N. chiamate	Dimensione	Costo per chiamata	Costo del livello
0	1	n	n^3	n^3
1	4	$n/2$	$(n/2)^3$	$n^3/2$
2	4^2	$n/2^2$	$(n/2^2)^3$	$n^3/2^2$
i	4^i	$n/2^i$	$(n/2^i)^3$	$n^3/2^i$
k	4^k	1	1	n^2

$$T(n) = n^3 \sum_{i=0}^{k-1} \left(\frac{1}{2}\right)^i + n^2 \leq 2n^3 + n^2 = \mathcal{O}(n^3).$$

Poiché la sola radice costa n^3 , vale anche $T(n) = \Omega(n^3)$; dunque $T(n) = \Theta(n^3)$. Dominano i primi livelli.

Tre comportamenti, una sola idea

Ricorrenza	Complessità	Interpretazione
$4T(n/2) + n$	$\Theta(n^2)$	dominano gli ultimi livelli
$4T(n/2) + n^2$	$\Theta(n^2 \log n)$	tutti i livelli costano uguale
$4T(n/2) + n^3$	$\Theta(n^3)$	dominano i primi livelli

Verso il Master Theorem

La complessità dipende dal confronto fra la crescita del numero di sottoproblemi e il costo del lavoro non ricorsivo.

Algoritmi e Strutture Dati

Analisi di algoritmi Ricorrenze, metodo di sostituzione

Alberto Montresor

Università di Trento

2026/09/10

This work is licensed under a Creative Commons
Attribution-ShareAlike 4.0 International License.



Metodo della sostituzione

Metodi per risolvere ricorrenze

- Analisi per livelli
- **Metodo di sostituzione, o per tentativi**
- Metodo dell'esperto, o delle ricorrenze comuni

Metodo di sostituzione

È un metodo in cui si cerca di “**indovinare**” (guess) una soluzione, in base alla propria esperienza, e si dimostra che questa soluzione è corretta tramite **induzione**.

Cerchiamo di indovinare

$$T(n) = \begin{cases} T(\lfloor n/2 \rfloor) + n & n > 1 \\ 1 & n \leq 1 \end{cases}$$

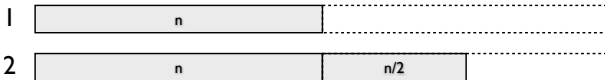
Cerchiamo di indovinare

$$T(n) = \begin{cases} T(\lfloor n/2 \rfloor) + n & n > 1 \\ 1 & n \leq 1 \end{cases}$$



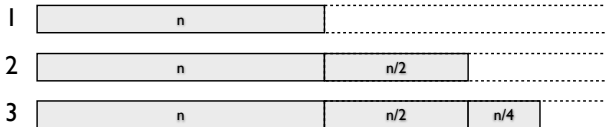
Cerchiamo di indovinare

$$T(n) = \begin{cases} T(\lfloor n/2 \rfloor) + n & n > 1 \\ 1 & n \leq 1 \end{cases}$$



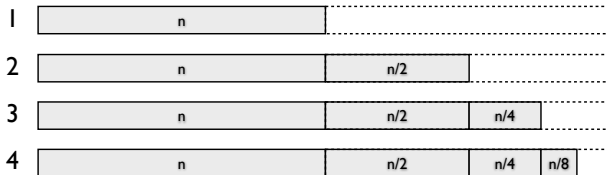
Cerchiamo di indovinare

$$T(n) = \begin{cases} T(\lfloor n/2 \rfloor) + n & n > 1 \\ 1 & n \leq 1 \end{cases}$$



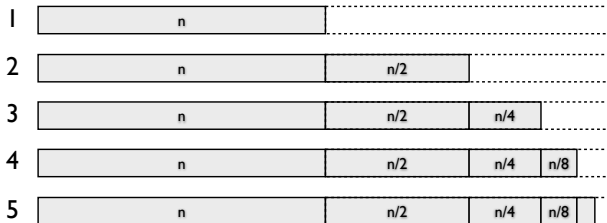
Cerchiamo di indovinare

$$T(n) = \begin{cases} T(\lfloor n/2 \rfloor) + n & n > 1 \\ 1 & n \leq 1 \end{cases}$$



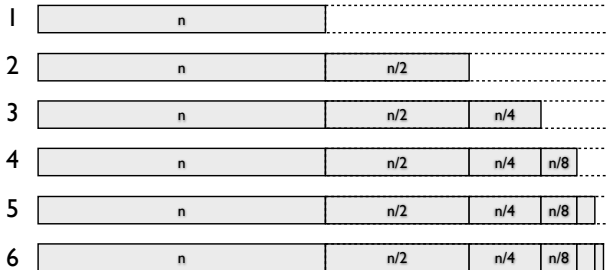
Cerchiamo di indovinare

$$T(n) = \begin{cases} T(\lfloor n/2 \rfloor) + n & n > 1 \\ 1 & n \leq 1 \end{cases}$$



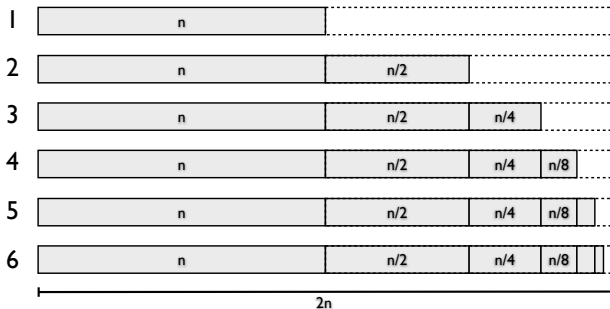
Cerchiamo di indovinare

$$T(n) = \begin{cases} T(\lfloor n/2 \rfloor) + n & n > 1 \\ 1 & n \leq 1 \end{cases}$$



Cerchiamo di indovinare

$$T(n) = \begin{cases} T(\lfloor n/2 \rfloor) + n & n > 1 \\ 1 & n \leq 1 \end{cases}$$



Cerchiamo di indovinare

$$T(n) = \begin{cases} T(\lfloor n/2 \rfloor) + n & n > 1 \\ 1 & n \leq 1 \end{cases}$$

$$T(n) = n \cdot \sum_{i=0}^{\log n} (1/2)^i \leq n \cdot \sum_{i=0}^{\infty} (1/2)^i = n \cdot \frac{1}{1 - \frac{1}{2}} = 2n$$

Serie geometrica decrescente infinita

$$\forall x, |x| < 1 : \sum_{i=0}^{\infty} x^i = \frac{1}{1 - x}$$

Limite superiore

$$T(n) = \begin{cases} T(\lfloor n/2 \rfloor) + n & n > 1 \\ 1 & n \leq 1 \end{cases} \quad \text{Tentativo: } T(n) = \mathcal{O}(n)$$
$$\exists c > 0, \exists m \geq 0 : T(n) \leq cn, \forall n \geq m$$

- **Caso base:** Dimostriamo la disequazione per $T(1)$

$$T(1) = 1 \stackrel{?}{\leq} 1 \cdot c \Leftrightarrow \forall c \geq 1$$

Limite superiore

$$T(n) = \begin{cases} T(\lfloor n/2 \rfloor) + n & n > 1 \\ 1 & n \leq 1 \end{cases} \quad \text{Tentativo: } T(n) = \mathcal{O}(n)$$

$$\exists c > 0, \exists m \geq 0 : T(n) \leq cn, \forall n \geq m$$

- **Ipotesi induttiva:** $\forall k < n : T(k) \leq ck$.
- **Passo di induzione:** Dimostriamo la disequazione per $T(n)$

$$T(n) = T(\lfloor n/2 \rfloor) + n$$

$$\leq c\lfloor n/2 \rfloor + n$$

$$\leq cn/2 + n$$

$$= (c/2 + 1)n$$

?

$$\leq cn$$

$$\Leftrightarrow c/2 + 1 \leq c \Leftrightarrow c \geq 2$$

Sostituzione

Intero inferiore

Passo algebrico

Obiettivo

Risultato finale

Limite superiore

$$T(n) = \begin{cases} T(\lfloor n/2 \rfloor) + n & n > 1 \\ 1 & n \leq 1 \end{cases} \quad \text{Tentativo: } T(n) = \mathcal{O}(n)$$

$\exists c > 0, \exists m \geq 0 : T(n) \leq cn, \forall n \geq m$

- Abbiamo provato che $T(n) \leq cn$
 - Nel caso base: $c \geq 1$
 - Nel passo induttivo: $c \geq 2$
 - Un valore c che rispetta entrambe le disequazioni è $c = 2$
- Questo vale per $n = 1$, e per tutti i valori di n seguenti
 - Quindi $m = 1$

Abbiamo quindi provato che $T(n) = \mathcal{O}(n)$

Limite inferiore

$$T(n) = \begin{cases} T(\lfloor n/2 \rfloor) + n & n > 1 \\ 1 & n \leq 1 \end{cases}$$

Tentativo: $T(n) = \Omega(n)$

$$\exists d > 0, \exists m \geq 0 : T(n) \geq dn, \forall n \geq m$$

- **Caso base:** Dimostriamo la disequazione per $T(1)$

$$T(1) = 1 \stackrel{?}{\geq} 1 \cdot d \Leftrightarrow \forall d \leq 1$$

Limite inferiore

$$T(n) = \begin{cases} T(\lfloor n/2 \rfloor) + n & n > 1 \\ 1 & n \leq 1 \end{cases}$$

Tentativo: $T(n) = \Omega(n)$

$$\exists d > 0, \exists m \geq 0 : T(n) \geq dn, \forall n \geq m$$

- Ipotesi induttiva:** $\forall k < n : T(k) \geq dk$.
- Passo di induzione:** Dimostriamo la disequazione per $T(n)$

$$T(n) = T(\lfloor n/2 \rfloor) + n$$

$$\geq d \lfloor n/2 \rfloor + n$$

$$\geq dn/2 - 1 + n$$

$$= \left(\frac{d}{2} - \frac{1}{n} + 1 \right) n \stackrel{?}{\geq} dn$$

$$\Leftrightarrow \frac{d}{2} - \frac{1}{n} + 1 \geq d \Leftrightarrow d \leq 2 - 2/n$$

Sostituzione

Intero inferiore

Passo algebrico

Risultato finale

Limite inferiore

$$T(n) = \begin{cases} T(\lfloor n/2 \rfloor) + n & n > 1 \\ 1 & n \leq 1 \end{cases}$$

Tentativo: $T(n) = \Omega(n)$

$$\exists d > 0, \exists m \geq 0 : T(n) \geq dn, \forall n \geq m$$

- Abbiamo provato che $T(n) \geq dn$
 - Nel caso base: $d \leq 1$
 - Nel passo induttivo: $d \leq 2 - \frac{2}{n}$
 - Un valore d che rispetta entrambe le disequazioni, per ogni valore di $n \geq 2$, è $d = 1$
- Questo vale per $n = 2$, e per tutti i valori di n seguenti
 - Quindi $m = 2$

Abbiamo quindi provato che
 $T(n) = \Omega(n)$

$T(n) = \mathcal{O}(n) \wedge T(n) = \Omega(n) \Leftrightarrow$
 $T(n) = \Theta(n)$

Limite inferiore

$$T(n) = \begin{cases} T(\lfloor n/2 \rfloor) + n & n > 1 \\ 1 & n \leq 1 \end{cases}$$

Tentativo: $T(n) = \Omega(n)$

$$\exists d > 0, \exists m \geq 0 : T(n) \geq dn, \forall n \geq m$$

- É possibile dimostrare che $T(n) = \Omega(n)$ in maniera molto più semplice, senza fare nemmeno ricorso all'ipotesi induttiva.

$$T(n) = T(\lfloor n/2 \rfloor) + n \stackrel{?}{\geq} n \geq dn$$

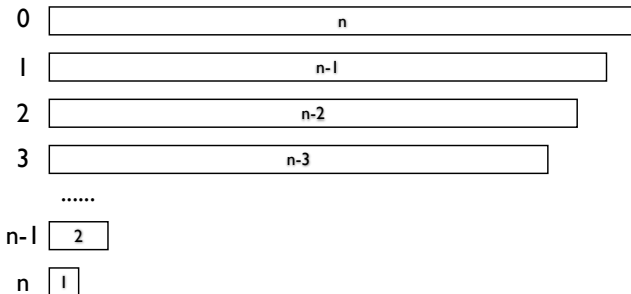
- L'ultima equazione è vera per $d \leq 1$, condizione identica a quella del caso base

Cosa succede se si sbaglia l'intuizione

$$T(n) = \begin{cases} T(n-1) + n & n > 1 \\ 1 & n \leq 1 \end{cases}$$

Cosa succede se si sbaglia l'intuizione

$$T(n) = \begin{cases} T(n-1) + n & n > 1 \\ 1 & n \leq 1 \end{cases}$$



Cosa succede se si sbaglia l'intuizione

$$T(n) = \begin{cases} T(n-1) + n & n > 1 \\ 1 & n \leq 1 \end{cases}$$

$$T(n) = \sum_{i=1}^n i = \frac{n(n+1)}{2} = \Theta(n^2)$$

Cosa succede se si sbaglia l'intuizione

$$T(n) = \begin{cases} T(n-1) + n & n > 1 \\ 1 & n \leq 1 \end{cases} \quad \text{Tentativo sbagliato: } T(n) = \mathcal{O}(n)$$

$$\exists c > 0, \exists m \geq 0 : T(n) \leq cn, \forall n \geq m$$

- **Ipotesi induttiva:** $\forall k < n : T(k) \leq ck$.
- **Passo di induzione:** Dimostriamo la disequazione per $T(n)$

$$T(n) = T(n-1) + n$$

$$\leq c(n-1) + n$$

$$\leq (c+1)n - c$$

?

$$\leq cn$$

$$\Rightarrow n - c \leq 0$$

$$\Rightarrow c \geq n$$

Sostituzione

Passo algebrico

Obiettivo

Semplificando cn

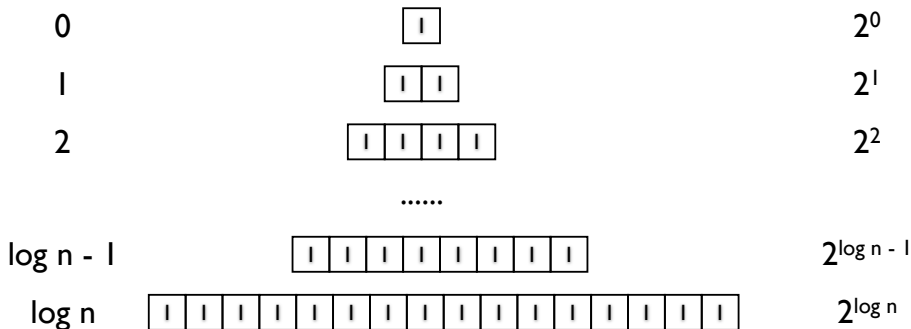
c non è costante

Difficoltà matematica – Limite superiore

$$T(n) = \begin{cases} T(\lfloor n/2 \rfloor) + T(\lceil n/2 \rceil) + 1 & n > 1 \\ 1 & n \leq 1 \end{cases}$$

Difficoltà matematica – Limite superiore

$$T(n) = \begin{cases} T(\lfloor n/2 \rfloor) + T(\lceil n/2 \rceil) + 1 & n > 1 \\ 1 & n \leq 1 \end{cases}$$



Difficoltà matematica – Limite superiore

$$T(n) = \begin{cases} T(\lfloor n/2 \rfloor) + T(\lceil n/2 \rceil) + 1 & n > 1 \\ 1 & n \leq 1 \end{cases}$$

$$T(n) = \sum_{i=0}^{\log n} 2^i = 1 + 2 + \dots + n/4 + n/2 + n = \mathcal{O}(n)$$

Difficoltà matematica – Limite superiore

$$T(n) = \begin{cases} T(\lfloor n/2 \rfloor) + T(\lceil n/2 \rceil) + 1 & n > 1 \\ 1 & n \leq 1 \end{cases}$$

Tentativo: $T(n) = \mathcal{O}(n)$
 $\exists c > 0, \exists m \geq 0 :$
 $T(n) \leq cn, \forall n \geq m$

- **Ipotesi induttiva:** $\forall k < n : T(k) \leq ck$.
- **Passo di induzione:** Dimostriamo la disequazione per $T(n)$

$$\begin{aligned} T(n) &= T(\lfloor n/2 \rfloor) + T(\lceil n/2 \rceil) + 1 \\ &\leq c\lfloor n/2 \rfloor + c\lceil n/2 \rceil + 1 \\ &= cn + 1 \\ &\stackrel{?}{\leq} cn \\ &\Rightarrow 1 \leq 0 \end{aligned}$$

Sostituzione

Passo algebrico

Obiettivo

Impossibile

Difficoltà matematica – Limite superiore

$$T(n) = \begin{cases} T(\lfloor n/2 \rfloor) + T(\lceil n/2 \rceil) + 1 & n > 1 \\ 1 & n \leq 1 \end{cases}$$

Tentativo: $T(n) = \mathcal{O}(n)$

$\exists c > 0, \exists m \geq 0 :$

$T(n) \leq cn, \forall n \geq m$

Cosa succede?

- Il tentativo è corretto...
- ma non riusciamo a dimostrarlo per **un termine di ordine inferiore**

$$cn + 1 \leq cn$$

Difficoltà matematica – Limite superiore

$$T(n) = \begin{cases} T(\lfloor n/2 \rfloor) + T(\lceil n/2 \rceil) + 1 & n > 1 \\ 1 & n \leq 1 \end{cases}$$

Tentativo: $T(n) = \mathcal{O}(n)$

$\exists c > 0, \exists m \geq 0 :$
 $T(n) \leq cn, \forall n \geq m$

- **Ipotesi induttiva più stretta:** $\exists b > 0, \forall k < n : T(k) \leq ck - b.$
- **Passo di induzione:** Dimostriamo la disequazione per $T(n)$:

$$\begin{aligned} T(n) &= T(\lfloor n/2 \rfloor) + T(\lceil n/2 \rceil) + 1 \\ &\leq c\lfloor n/2 \rfloor - b + c\lceil n/2 \rceil - b + 1 \\ &= cn - 2b + 1 \\ &\stackrel{?}{\leq} cn - b \\ &\Rightarrow -2b + 1 \leq -b \\ &\Rightarrow b \geq 1 \end{aligned}$$

Sostituzione

Passo algebrico

Obiettivo

Eliminazione cn

Passo algebrico

Difficoltà matematica – Limite superiore

$$T(n) = \begin{cases} T(\lfloor n/2 \rfloor) + T(\lceil n/2 \rceil) + 1 & n > 1 \\ 1 & n \leq 1 \end{cases}$$

Tentativo: $T(n) = \mathcal{O}(n)$

$\exists c > 0, \exists m \geq 0 :$
 $T(n) \leq cn, \forall n \geq m$

- **Caso base:** Dimostriamo la disequazione per $T(1)$

$$T(1) = 1 \stackrel{?}{\leq} 1 \cdot c - b \Leftrightarrow \forall c \geq b + 1$$

Difficoltà matematica – Limite superiore

$$T(n) = \begin{cases} T(\lfloor n/2 \rfloor) + T(\lceil n/2 \rceil) + 1 & n > 1 \\ 1 & n \leq 1 \end{cases}$$

Tentativo: $T(n) = \mathcal{O}(n)$

$\exists c > 0, \exists m \geq 0 :$
 $T(n) \leq cn, \forall n \geq m$

- Abbiamo provato che $T(n) \leq cn - b \leq cn$
 - Nel passo induttivo: $\forall b \geq 1, \forall c$
 - Nel caso base: $\forall c \geq b + 1$
 - Una coppia di valori b, c che rispettano queste disequazioni sono $b = 1, c = 2$
- Questo vale per $n = 1$, e per tutti i valori di n seguenti
 - Quindi $m = 1$

Abbiamo quindi provato che $T(n) = \mathcal{O}(n)$

Difficoltà matematica – Limite inferiore

$$T(n) = \begin{cases} T(\lfloor n/2 \rfloor) + T(\lceil n/2 \rceil) + 1 & n > 1 \\ 1 & n \leq 1 \end{cases}$$

Tentativo: $T(n) = \Omega(n)$
 $\exists d > 0, \exists m \geq 0 :$
 $T(n) \geq dn, \forall n \geq m$

- **Passo di induzione:** Dimostriamo la disequazione per $T(n)$

$$\begin{aligned} T(n) &= T(\lfloor n/2 \rfloor) + T(\lceil n/2 \rceil) + 1 \\ &\geq d\lfloor n/2 \rfloor + d\lceil n/2 \rceil + 1 \\ &= dn + 1 \stackrel{?}{\geq} dn \end{aligned}$$

Sostituzione

Vero per ogni d

- **Caso base:** Dimostriamo la disequazione per $T(1)$

$$T(1) = 1 \geq d \cdot 1 \Leftrightarrow d \leq 1$$

Abbiamo quindi provato che $T(n) = \Omega(n)$

Problemi con i casi base

$$T(n) = \begin{cases} 2T(\lfloor n/2 \rfloor) + n & n > 1 \\ 1 & n \leq 1 \end{cases}$$

0

n

1

n/2	n/2
-----	-----

2

n/4	n/4	n/4	n/4
-----	-----	-----	-----

.....

log n - 1

2	2	2	2	2	2	2	2	2	2
---	---	---	---	---	---	---	---	---	---

log n

1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

Problemi con i casi base

$$T(n) = \begin{cases} 2T(\lfloor n/2 \rfloor) + n & n > 1 \\ 1 & n \leq 1 \end{cases}$$

$$T(n) = \mathcal{O}(n \log n)$$

Problemi con i casi base

$$T(n) = \begin{cases} 2T(\lfloor n/2 \rfloor) + n & n > 1 \\ 1 & n \leq 1 \end{cases}$$

Tentativo: $T(n) = \mathcal{O}(n \log n)$
 $\exists c > 0, \exists m \geq 0 :$
 $T(n) \leq cn \log n, \forall n \geq m$

- **Ipotesi induttiva:** $\exists c > 0, \forall k < n : T(k) \leq ck \log k$.
- **Passo di induzione:** Dimostriamo la disequazione per $T(n)$:

$$\begin{aligned} T(n) &= 2T(\lfloor n/2 \rfloor) + n \\ &\leq 2c\lfloor n/2 \rfloor \log \lfloor n/2 \rfloor + n \\ &\leq 2cn/2 \log n/2 + n \\ &= cn(\log n - 1) + n \\ &= cn \log n - cn + n \\ &\stackrel{?}{\leq} cn \log n \end{aligned}$$

Sostituzione

Intero inferiore

Passo algebrico

Passo algebrico

Obiettivo

Problemi con i casi base

$$T(n) = \begin{cases} 2T(\lfloor n/2 \rfloor) + n & n > 1 \\ 1 & n \leq 1 \end{cases}$$

Tentativo: $T(n) = \mathcal{O}(n \log n)$
 $\exists c > 0, \exists m \geq 0 :$
 $T(n) \leq cn \log n, \forall n \geq m$

- **Ipotesi induttiva:** $\exists c > 0, \forall k < n : T(k) \leq ck \log k$.
- **Passo di induzione:** Dimostriamo la disequazione per $T(n)$:

$$\begin{aligned} T(n) &\leq cn \log n - cn + n \stackrel{?}{\leq} cn \log n \\ &\Rightarrow -cn + n \leq 0 \\ &\Rightarrow c \geq 1 \end{aligned}$$

Obiettivo

Eliminazione $cn \log n$

Passo algebrico

Problemi con i casi base

$$T(n) = \begin{cases} 2T(\lfloor n/2 \rfloor) + n & n > 1 \\ 1 & n \leq 1 \end{cases}$$

Tentativo: $T(n) = \mathcal{O}(n \log n)$
 $\exists c > 0, \exists m \geq 0 :$
 $T(n) \leq cn \log n, \forall n \geq m$

- **Caso base:** Dimostriamo la disequazione per $T(1)$

$$T(1) = 1 \stackrel{?}{\leq} 1 \cdot c \log 1 = 0 \Rightarrow 1 \not\leq 0$$

Problemi con i casi base

$$T(n) = \begin{cases} 2T(\lfloor n/2 \rfloor) + n & n > 1 \\ 1 & n \leq 1 \end{cases}$$

Tentativo: $T(n) = \mathcal{O}(n \log n)$

$\exists c > 0, \exists m \geq 0 :$

$T(n) \leq cn \log n, \forall n \geq m$

Cosa succede?

- È falso, ma non è un problema: non a caso si chiama notazione asintotica.
- Il valore iniziale di m lo possiamo scegliere noi

Problemi con i casi base

$$T(n) = \begin{cases} 2T(\lfloor n/2 \rfloor) + n & n > 1 \\ 1 & n \leq 1 \end{cases}$$

Tentativo: $T(n) = \mathcal{O}(n \log n)$
 $\exists c > 0, \exists m \geq 0 :$
 $T(n) \leq cn \log n, \forall n \geq m$

- **Caso base:** Dimostriamo la disequazione per $T(2)$, $T(3)$:

$$T(2) = 2T(\lfloor 2/2 \rfloor) + 2 = 4 \leq 1 \cdot c \cdot 2 \log 2 \Leftrightarrow c \geq 2$$

$$T(3) = 2T(\lfloor 3/2 \rfloor) + 3 = 5 \leq 1 \cdot c \cdot 3 \log 3 \Leftrightarrow c \geq \frac{5}{3 \log 3}$$

$$T(4) = 2T(\lfloor 4/2 \rfloor) + 4 = 2T(\lfloor 2 \rfloor) + 4$$

- Non è necessario provare la terza disequazione, in quanto viene espressa in base a casi base diversi da $T(1)$ che sono già stati dimostrati e quindi possono costituire la base della nostra induzione.

Problemi con i casi base

$$T(n) = \begin{cases} 2T(\lfloor n/2 \rfloor) + n & n > 1 \\ 1 & n \leq 1 \end{cases}$$

Tentativo: $T(n) = \mathcal{O}(n \log n)$
 $\exists c > 0, \exists m \geq 0 :$
 $T(n) \leq cn \log n, \forall n \geq m$

- Abbiamo provato che $T(n) \leq cn \log n$
 - Nel passo induttivo: $\forall c \geq 1$
 - Nel caso base: $\forall c \geq 2, c \geq \frac{5}{3 \log 3}$
 - Visto che sono tutte disequazioni con il segno $\geq$, è sufficiente utilizzare un valore $c \geq \max\{1, 2, \frac{5}{3 \log 3}\}$
- Questo vale per $n = 2, n = 3$, e per tutti i valori di n seguenti
 - Quindi $m = 2$

together now!

$$T(n) = \begin{cases} 9T(\lfloor n/3 \rfloor) + n & n > 1 \\ 1 & n \leq 1 \end{cases} \quad \text{Tentativo: } T(n) = \mathcal{O}(n^2)$$

$$\exists c > 0, \exists m \geq 0 : T(n) \leq cn^2, \forall n \geq m$$

- **Ipotesi induttiva:** $\exists c > 0 : T(k) \leq ck^2, \forall k < n$
- **Passo induttivo:** Dimostriamo la disequazione per $T(n)$

$$\begin{aligned} T(n) &= 9T(\lfloor n/3 \rfloor) + n \\ &\leq 9c(\lfloor n/3 \rfloor)^2 + n \\ &\leq 9c(n^2/9) + n \\ &= cn^2 + n \\ &\not\leq cn^2 \end{aligned}$$

Sostituzione

Limite inferiore

Passo algebrico

Falso

together now!

$$T(n) = \begin{cases} 9T(\lfloor n/3 \rfloor) + n & n > 1 \\ 1 & n \leq 1 \end{cases} \quad \text{Tentativo: } T(n) = \mathcal{O}(n^2)$$

$$\exists c > 0, \exists m \geq 0 : T(n) \leq cn^2, \forall n \geq m$$

- **Ipotesi induttiva:** $\exists c > 0 : T(k) \leq c(k^2 - k), \forall k < n$
- **Passo induttivo:** Dimostriamo la disequazione per $T(n)$

$$\begin{aligned} T(n) &= 9T(\lfloor n/3 \rfloor) + n \\ &\leq 9c(\lfloor n/3 \rfloor^2 - \lfloor n/3 \rfloor) + n \\ &\leq cn^2 - 3cn + n \\ &\stackrel{?}{\leq} cn^2 - cn \\ &\Leftrightarrow c \geq \frac{1}{2} \end{aligned}$$

Sostituzione

Limite inferiore

Obiettivo

together now!

$$T(n) = \begin{cases} 9T(\lfloor n/3 \rfloor) + n & n > 1 \\ 1 & n \leq 1 \end{cases} \quad \text{Tentativo: } T(n) = \mathcal{O}(n^2)$$

$\exists c > 0, \exists m \geq 0 : T(n) \leq cn^2, \forall n \geq m$

- **Ipotesi induttiva:** $\exists c > 0 : T(k) \leq c(k^2 - k), \forall k < n$
- **Passo base:**
 - $T(1) = 1 \leq c(1^2 - 1) = 0$, falso
 - $T(2) = 9T(0) + 2 = 11 \leq c(4 - 2) \Leftrightarrow c \geq 11/2$
 - $T(3) = 9T(1) + 3 = 12 \leq c(9 - 3) \Leftrightarrow c \geq 12/6$
 - $T(4) = 9T(1) + 4 = 13 \leq c(16 - 4) \Leftrightarrow c \geq 13/12$
 - $T(5) = 9T(1) + 5 = 14 \leq c(25 - 5) \Leftrightarrow c \geq 14/20$
 - $T(6) = 9T(2) + 6$
- Non è necessario andare oltre, perchè $T(6)$ dipende da $T(2)$ che è già stato dimostrato

together now!

$$T(n) = \begin{cases} 9T(\lfloor n/3 \rfloor) + n & n > 1 \\ 1 & n \leq 1 \end{cases} \quad \text{Tentativo: } T(n) = \mathcal{O}(n^2)$$

$\exists c > 0, \exists m \geq 0 : T(n) \leq cn^2, \forall n \geq m$

- Parametri:

- $c \geq \max \left\{ \frac{1}{2}, \frac{11}{2}, \frac{12}{6}, \frac{13}{12}, \frac{14}{20} \right\}$
- $m = 2$

- Notare che l'esempio combina le due difficoltà insieme, ma è artificiale:

- Se avessimo scelto come ipotesi più stretta $T(n) \leq cn^2 - bn$, il problema sui casi base non si sarebbe posto

Riassumendo

Metodo di sostituzione

- Si “**indovina**” una possibile soluzione e si formula un’ipotesi induttiva
- Si **sostituisce** nella ricorrenza le espressioni $T(\cdot)$, utilizzando l’ipotesi induttiva
- Si **dimostra** che la soluzione è valida anche per il caso base

Attenzione

- Ad ipotizzare soluzioni troppo “strette”
- Ad alcuni casi particolari che richiedono alcune astuzie matematiche
- Attenzione ai casi base: il logaritmo può complicare le cose

Algoritmi e Strutture Dati

Analisi di algoritmi Master Theorem e altre ricorrenze

Alberto Montresor

Università di Trento

2026/09/10

This work is licensed under a Creative Commons
Attribution-ShareAlike 4.0 International License.



Teoremi degli esperti

Tre metodi complementari

- Analisi per livelli
- Metodo di sostituzione, o per tentativi
- **Teoremi degli esperti, o delle ricorrenze comuni**

Ricorrenze divide-et-impera

Molti algoritmi generano ricorrenze della forma

$$T(n) = aT(n/b) + f(n),$$

dove a è il numero di sottoproblemi, n/b la loro dimensione e $f(n)$ il lavoro di divisione e ricombinazione.

Master Theorem, versione ridotta

Teorema

Siano $a \geq 1$ e $b > 1$ costanti, $c, d > 0$ e $\beta \geq 0$. Per

$$T(n) = \begin{cases} d & n \leq 1, \\ aT(n/b) + cn^\beta & n > 1, \end{cases}$$

considerata dapprima per n potenza di b , posto $\alpha = \log_b a$ vale

$$T(n) = \begin{cases} \Theta(n^\alpha) & \alpha > \beta, \\ \Theta(n^\alpha \log n) & \alpha = \beta, \\ \Theta(n^\beta) & \alpha < \beta. \end{cases}$$

Dimostrazione: analisi per livelli

Poniamo $n = b^k$, quindi $k = \log_b n$.

Livello	N. chiamate	Dimensione	Costo per chiamata	Costo del livello
0	1	b^k	$cb^{k\beta}$	$cb^{k\beta}$
1	a	b^{k-1}	$cb^{(k-1)\beta}$	$acb^{(k-1)\beta}$
2	a^2	b^{k-2}	$cb^{(k-2)\beta}$	$a^2cb^{(k-2)\beta}$
i	a^i	b^{k-i}	$cb^{(k-i)\beta}$	$a^i cb^{(k-i)\beta}$
$k-1$	a^{k-1}	b	cb^β	$a^{k-1}cb^\beta$
k	a^k	1	d	da^k

Poiché $a = b^\alpha$, abbiamo $a^k = n^\alpha$ e $b^{k\beta} = n^\beta$.

Dimostrazione: una somma geometrica

Poniamo

$$q = \frac{a}{b^\beta} = b^{\alpha-\beta}.$$

Il costo del livello interno i è $cn^\beta q^i$; sommando livelli interni e foglie otteniamo

$$T(n) = dn^\alpha + cn^\beta \sum_{i=0}^{k-1} q^i.$$

Tutto dipende da q

$$\alpha > \beta$$

$$q > 1$$

il costo cresce

$$\alpha = \beta$$

$$q = 1$$

il costo è costante

$$\alpha < \beta$$

$$0 < q < 1$$

il costo diminuisce

Dimostrazione: $\alpha > \beta$ e $\alpha = \beta$

Caso 1: $\alpha > \beta$, quindi $q > 1$

$$q^{k-1} \leq \sum_{i=0}^{k-1} q^i = \frac{q^k - 1}{q - 1} \leq \frac{q^k}{q - 1}.$$

Poiché $q^k = n^{\alpha-\beta}$, i livelli interni costano $\Theta(n^\beta n^{\alpha-\beta}) = \Theta(n^\alpha)$; le foglie costano anch'esse dn^α .

Caso 2: $\alpha = \beta$, quindi $q = 1$

Tutti i k livelli interni costano cn^α :

$$T(n) = dn^\alpha + ckn^\alpha = n^\alpha(d + c \log_b n) = \Theta(n^\alpha \log n).$$

Dimostrazione: $\alpha < \beta$

Caso 3: $\alpha < \beta$, quindi $0 < q < 1$

$$1 \leq \sum_{i=0}^{k-1} q^i \leq \sum_{i=0}^{\infty} q^i = \frac{1}{1-q}.$$

I livelli interni costano dunque $\Theta(n^\beta)$.

- le foglie costano $dn^\alpha = \mathcal{O}(n^\beta)$;
- la sola radice costa cn^β ;
- pertanto $T(n) = \Theta(n^\beta)$.

La dimostrazione formalizza la stessa tricotomia osservata con l'analisi per livelli dei tre esempi $4T(n/2) + n^j$.

Versione ridotta: applicazioni

Ricorrenza	a	b	Confronto	Soluzione
$3T(n/2) + n$	3	2	$\log_2 3 > 1$	$\Theta(n^{\log_2 3})$
$3T(n/2) + 1$	3	2	$\log_2 3 > 0$	$\Theta(n^{\log_2 3})$
$3T(n/2) + n^2$	3	2	$\log_2 3 < 2$	$\Theta(n^2)$
$4T(n/2) + n^2$	4	2	$\log_2 4 = 2$	$\Theta(n^2 \log n)$

Tre algoritmi già incontrati

Karatsuba: $a = 3, b = 2, \beta = 1$; Merge Sort: $a = b = 2, \beta = 1$; ricerca binaria: $a = 1, b = 2, \beta = 0$.

Potenze di b e arrotondamenti

Perché assumiamo $n = b^k$?

L'assunzione rende esatta la profondità $k = \log_b n$ e semplifica i calcoli.

- Nella realtà, le dimensioni dei sottoproblemi richiedono interi inferiori o superiori.
- Se T è non decrescente, possiamo racchiudere n fra due potenze consecutive di b :

$$b^{\lfloor \log_b n \rfloor} \leq n \leq b^{\lceil \log_b n \rceil} < bn.$$

- Il padding cambia la dimensione per meno del fattore costante b e la profondità per al più un livello: l'ordine asintotico resta invariato.

Master Theorem, versione estesa

Teorema

Sia $T(n) = aT(n/b) + f(n)$, con $a \geq 1$, $b > 1$ costanti, $f(n)$ asintoticamente positiva e $\alpha = \log_b a$.

- 1 Se $f(n) = \mathcal{O}(n^{\alpha-\epsilon})$ per qualche $\epsilon > 0$, allora $T(n) = \Theta(n^\alpha)$.
- 2 Se $f(n) = \Theta(n^\alpha)$, allora $T(n) = \Theta(n^\alpha \log n)$.
- 3 Se $f(n) = \Omega(n^{\alpha+\epsilon})$ per qualche $\epsilon > 0$ e $af(n/b) \leq cf(n)$ definitivamente, per qualche $c < 1$, allora $T(n) = \Theta(f(n))$.

Nei casi 1 e 3 serve una **distanza polinomiale**; nel caso 3 serve anche la condizione di regolarità.

Versione estesa: applicazioni

Ricorrenza	α	Caso	Soluzione
$4T(n/2) + n \log n$	2	1	$\Theta(n^2)$
$T(2n/3) + 1$	0	2	$\Theta(\log n)$
$3T(n/4) + n \log n$	$\log_4 3$	3	$\Theta(n \log n)$

Per il terzo esempio la regolarità vale, perché

$$3f(n/4) = \frac{3}{4}n(\log n - \log 4) \leq \frac{3}{4}n \log n = \frac{3}{4}f(n).$$

Che cosa significa un tentativo fallito?

Ricorrenza: $T(n) = 2T(n/2) + n \log n$, quindi $\alpha = 1$ e $f(n) = n^\alpha \log n$.

- Non è il caso 1:

$$\frac{n^\alpha \log n}{n^{\alpha-\epsilon}} = n^\epsilon \log n \longrightarrow \infty.$$

- Non è il caso 2:

$$\frac{n^\alpha \log n}{n^\alpha} = \log n \longrightarrow \infty.$$

- Non è il caso 3:

$$\frac{n^\alpha \log n}{n^{\alpha+\epsilon}} = \frac{\log n}{n^\epsilon} \longrightarrow 0.$$

Il fattore $\log n$ crea una distanza più che costante, ma meno che polinomiale: resta nello spazio fra i tre casi.

Master Theorem con fattori logaritmici

Estensione

Sia

$$T(n) = aT(n/b) + \Theta(n^\beta \log^k n),$$

con $k \in \mathbb{R}$ e termine non ricorsivo positivo definitivamente. Posto $\alpha = \log_b a$:

$$T(n) = \begin{cases} \Theta(n^\alpha) & \beta < \alpha, \\ \Theta(n^\beta \log^k n) & \beta > \alpha, \\ \Theta(n^\alpha \log^{k+1} n) & \beta = \alpha, k > -1, \\ \Theta(n^\alpha \log \log n) & \beta = \alpha, k = -1, \\ \Theta(n^\alpha) & \beta = \alpha, k < -1. \end{cases}$$

Fattori logaritmici: due esempi

$$2T(n/2) + n \log n$$

$\alpha = \beta = 1$ e $k = 1 > -1$, quindi

$$T(n) = \Theta(n \log^2 n).$$

$$2T(n/2) + n / \log n$$

$\alpha = \beta = 1$ e $k = -1$, quindi

$$T(n) = \Theta(n \log \log n).$$

Oltre il Master Theorem

Sottoproblemi di dimensioni diverse

Il Master Theorem non si applica, per esempio, a

$$T(n) = T(n/3) + T(2n/3) + n.$$

Teorema di Akra–Bazzi

Tratta molte ricorrenze divide-et-impera con sottoproblemi sbilanciati. Il suo enunciato è però troppo tecnico per gli obiettivi del corso.

Ricorrenze lineari di ordine costante

Teorema

Siano $a_1, \dots, a_h$ interi non negativi, non tutti nulli, $h \geq 1$ costante e $\beta \geq 0$.
Se

$$T(n) = \begin{cases} \Theta(1) & n \leq m, \\ \sum_{i=1}^h a_i T(n-i) + \Theta(n^\beta) & n > m, \end{cases}$$

con $m \geq h$ costante e $a = \sum_{i=1}^h a_i$, allora:

- se $a = 1$, $T(n) = \Theta(n^{\beta+1})$;
- se $a \geq 2$, $T(n) = \mathcal{O}(a^n)$.

Il secondo limite è generale e può non essere stretto: per Fibonacci dà $\mathcal{O}(2^n)$, mentre il limite preciso è $\Theta(\varphi^n)$.

Ricorrenze lineari: esempi

Ricorrenza	a	β	Risultato
$T(n) = T(n - 10) + n^2$	1	2	$\Theta(n^3)$
$T(n) = T(n - 1) + n$	1	1	$\Theta(n^2)$
$T(n) = 2T(n - 1) + 1$	2	0	$\mathcal{O}(2^n)$

Per $2T(n - 1) + 1$, l'albero delle chiamate fornisce anche il limite inferiore corrispondente: $T(n) = \Theta(2^n)$.

Esercizio

Siano

- $T(n) = 7T(n/2) + n^2$ la funzione di costo di un algoritmo A ;
- $T'(n) = aT'(n/4) + n^2$ la funzione di costo di un algoritmo A' .

Qual è il massimo valore intero di a che rende A' asintoticamente più veloce di A ?

Esercizio – Soluzione

$$T(n) = \Theta(n^{\log_2 7}) = \Theta(n^{\log_4 49}).$$

Costo di A'

- $a < 16$: $T'(n) = \Theta(n^2)$;
- $a = 16$: $T'(n) = \Theta(n^2 \log n)$;
- $a > 16$: $T'(n) = \Theta(n^{\log_4 a})$.

Per essere strettamente più veloce serve $\log_4 a < \log_4 49$, cioè $a < 49$. Il massimo valore intero è quindi $a = 48$.

Algoritmi e strutture dati

Analisi di funzioni Back to algorithms!

Alberto Montresor

Università di Trento

2026/09/10

This work is licensed under a Creative Commons
Attribution-ShareAlike 4.0 International License.



Sommario

- 1 Ricorrenze
 - Introduzione
 - Analisi per livelli
 - Metodo della sostituzione
 - Teoremi degli esperti
 - Altre ricorrenze comuni

- 2 Back to algorithms!

Complessità della Versione 1

```

int maxsum1(int[] A, int n) {
    int maxSoFar = 0;
    for (int i=0; i < n; i++) {
        for (int j=i; j < n; j++) {
            int sum = 0;
            for (int k=i; k <= j; k++) {
                sum = sum + A[k];
            }
            maxSoFar = max(maxSoFar, sum);
        }
    }
    return maxSoFar;
}

```

La complessità dell'algoritmo può essere approssimata come segue (contando il numero di esecuzioni della riga più interna)

$$T(n) = \sum_{i=0}^{n-1} \sum_{j=i}^{n-1} (j - i + 1)$$

Complessità della Versione 1 - $\mathcal{O}(n^3)$

Vogliamo provare che $T(n) = \mathcal{O}(n^3)$, i.e.

$$\exists c_2 > 0, \exists m \geq 0 : T(n) \leq c_2 n^3, \forall n \geq m$$

$$\begin{aligned} T(n) &= \sum_{i=0}^{n-1} \sum_{j=i}^{n-1} (j - i + 1) \\ &\leq \sum_{i=0}^{n-1} \sum_{j=i}^{n-1} n \leq \sum_{i=0}^{n-1} \sum_{j=0}^{n-1} n \\ &= \sum_{i=0}^{n-1} n^2 = n^3 \leq c_2 n^3 \end{aligned}$$

Questa disequazione è vera per $n \geq m = 0$ e $c_2 \geq 1$.

Complessità della Versione 1 - $\Omega(n^3)$

Vogliamo provare che $T(n) = \Omega(n^3)$.

Assumiamo per semplicità che n sia multiplo di 4.

$$\begin{aligned}
 T(n) &= \sum_{i=0}^{n-1} \sum_{j=i}^{n-1} (j - i + 1) \\
 &\geq \sum_{i=0}^{n/4-1} \sum_{j=i+n/2}^{i+3n/4-1} (j - i + 1) \\
 &\geq \frac{n}{4} \cdot \frac{n}{4} \cdot \frac{n}{2} = \frac{n^3}{32} \geq c_1 n^3.
 \end{aligned}$$

L'ultima disequazione è vera per $c_1 \leq 1/32$.

Complessità della versione 2

```

int maxsum2(int[] A, int n) {
    int maxSoFar = 0;
    for (int i=0; i < n; i++) {
        int sum = 0;
        for (int j=i; j < n; j++) {
            sum = sum + A[j];
            maxSoFar = max(maxSoFar, sum);
        }
    }
    return maxSoFar;
}

```

La complessità di questo algoritmo può essere approssimata come segue (stiamo contando il numero di passi nel ciclo più interno)

$$T(n) = \sum_{i=0}^{n-1} n - i$$

Complessità della versione 2 - $\Theta(n^2)$

Vogliamo provare che $T(n) = \Theta(n^2)$.

$$\begin{aligned} T(n) &= \sum_{i=0}^{n-1} n - i \\ &= \sum_{i=1}^n i \\ &= \frac{n(n+1)}{2} = \Theta(n^2) \end{aligned}$$

Questo non richiede ulteriori dimostrazioni

Complessità della versione 3: divide

```

int maxsum_rec(int[] A, int i, int j) {
    if (i==j)
        return max(0, A[i]);
    int m = (i+j) / 2;
    int maxs = maxsum_rec(A, i, m);
    int maxd = maxsum_rec(A, m+1, j);
    int maxss = 0;
    int sum = 0;
    for (int k=m; k>=i; k--) {
        sum = sum+A[k];
        maxss = max(maxss, sum);
    }
}

```

Le due chiamate ricorsive risolvono i sottoproblemi; il ciclo calcola la migliore somma che termina nella metà sinistra.

Complessità della versione 3: combina

```

// Continua maxsum_rec()
int maxdd = 0;
sum = 0;
for (int k=m+1; k<=j; k++) {
    sum = sum+A[k];
    maxdd = max(maxdd, sum);
}
return max(max(maxs,maxd),
           maxss+maxdd);
}

int maxsum3(int[] A, int n) {
    return maxsum_rec(A, 0, n-1);
}

```

Il secondo ciclo calcola la migliore somma che inizia nella metà destra; il risultato confronta le due soluzioni ricorsive con quella che attraversa il punto medio.

Complessità della versione 3: analisi

Relazione di ricorrenza

$$T(n) = 2T(n/2) + n$$

Master Theorem, versione ridotta

$a = 2$, $b = 2$ e $\beta = 1$. Poiché

$$\alpha = \log_2 2 = 1 = \beta,$$

siamo nel secondo caso e quindi

$$T(n) = \Theta(n \log n).$$

Complessità della versione 4

```
int maxsum4(int A[], int n) {  
    int maxSoFar = 0;  
    int maxHere = 0;  
    for (int i=0; i < n; i++) {  
        maxHere = max(maxHere+A[i],0);  
        maxSoFar = max(maxSoFar,maxHere);  
    }  
    return maxSoFar;  
}
```

È facile vedere che la complessità di questa versione è $\Theta(n)$.